

확장판 · 개념 + 코드 + 코드 해석 + 실행 흐름

그래프부터 위상정렬까지

이 문서는 CLRS의 그래프 기초, BFS, DFS, 위상정렬을 처음 배우는 사람 기준으로 풀어쓴 확장판입니다. 단순 개념 요약에서 끝나지 않고, 코드가 왜 그렇게 생겼는지, 실행하면 내부에서 어떤 일이 벌어지는지까지 함께 설명합니다.

1. 이 문서를 어떻게 보면 좋은가

이번 문서는 세 층으로 구성되어 있습니다.

1. 개념: 그래프와 탐색이 무엇인지 이해합니다.
2. 구현: Python 코드로 실제로 어떻게 쓰는지 봅니다.
3. 해석: 코드 한 줄이 어떤 역할을 하는지 따라갑니다.

추천 공부법: 문서를 한 번 읽고 끝내지 말고, 중간중간 있는 예제 그래프를 종이에 그리면서 코드의 방문 순서를 직접 적어보면 훨씬 빨리 익습니다.

2. 그래프의 가장 기본 그림

그래프는 정점과 간선으로 이루어진 연결 구조입니다.

- 정점(vertex): 대상 하나
- 간선(edge): 두 대상을 잇는 연결
- 무방향 그래프: 양방향 연결
- 방향 그래프: 화살표 방향이 있는 연결

실생활 예시

사람을 정점으로 보고 친구 관계를 간선으로 보면 친구 네트워크를 그래프로 표현할 수 있습니다.

알고리즘 예시

과목을 정점으로 보고 선수 관계를 간선으로 보면 수강 순서를 그래프로 표현할 수 있습니다.

3. 방향 그래프와 무방향 그래프

3-1. 무방향 그래프

1 - 2 라고 적으면 1에서 2로 갈 수 있고, 2에서도 1로 갈 수 있습니다.

```

1 --- 2
|     |
3 --- 4

```

3-2. 방향 그래프

1 -> 2 라고 적으면 1에서 2로 가는 방향만 의미합니다.

```

1 -> 2 -> 4
\         ^
 -> 3 ----|

```

3-3. 둘의 차이가 중요한 이유

- 무방향 그래프는 "연결되어 있는가"가 중요할 때 자주 나옵니다.
- 방향 그래프는 "순서"나 "의존성"이 중요할 때 자주 나옵니다.

4. 그래프를 코드에 저장하는 두 가지 대표 방법

4-1. 인접 리스트

```

graph = {
  1: [2, 3],
  2: [1, 4],
  3: [1, 4],
  4: [2, 3]
}

```

각 정점마다 연결된 이웃 정점들의 목록을 갖고 있습니다.

- 장점: 메모리를 효율적으로 씁니다.
- 장점: 탐색 알고리즘과 잘 맞습니다.
- 단점: 연결 여부를 즉시 알기는 어렵습니다.

4-2. 인접 행렬

```

graph = [
  [0, 1, 1, 0],
  [1, 0, 0, 1],
  [1, 0, 0, 1],
  [0, 1, 1, 0]
]

```

행과 열의 번호를 정점으로 보고, 연결돼 있으면 1, 아니면 0으로 저장합니다.

4-3. 둘을 비교하면

방법	장점	단점	주로 쓰는 경우
인접 리스트	메모리 효율	직접 연결 여부 확인은 느릴 수 있음	대부분의 탐색 문제
인접 행렬	연결 여부 즉시 확인	메모리 사용량 큼	정점 수가 작을 때

5. 탐색을 왜 배우는가

그래프를 저장만 해서는 의미가 없습니다. 우리는 그래프 안을 돌아다니며 정보를 찾아야 합니다.

- 어떤 정점까지 갈 수 있는가
- 최단 거리로 가려면 몇 번 이동해야 하는가
- 사이클이 존재하는가
- 작업 순서를 어떻게 정해야 하는가

이때 기본이 되는 탐색이 BFS와 DFS입니다.

6. BFS의 핵심 감각

BFS는 시작점에서 가까운 정점부터 넓게 퍼져 나가며 방문합니다.

BFS를 한 줄로 말하면: **거리 1짜리를 다 보고, 그 다음 거리 2짜리를 보고, 그 다음 거리 3짜리를 보는 방식**입니다.

6-1. 큐가 필요한 이유

먼저 발견한 정점을 먼저 처리해야 가까운 정점부터 순서가 유지됩니다. 그래서 FIFO 구조인 큐를 씁니다.

6-2. BFS 기본 코드

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        now = queue.popleft()
        order.append(now)

        for nxt in graph[now]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)

    return order
```

6-3. 코드 해석

visited = set([start])

시작점을 이미 방문한 것으로 표시합니다. 같은 정점이 큐에 여러 번 들어가는 실수를 막습니다.

queue = deque([start])

탐색을 시작할 정점을 큐에 넣습니다. BFS는 큐에서 출발합니다.

order = []

실제로 어떤 순서로 방문했는지 기록합니다.

```
while queue:
```

큐에 아직 처리할 정점이 남아 있는 동안 계속 반복하겠다는 뜻입니다.

```
now = queue.popleft()
```

큐 맨 앞 정점을 꺼냅니다. 가장 먼저 들어온 정점이 먼저 나옵니다.

```
for nxt in graph[now]:
```

현재 정점과 연결된 모든 이웃을 확인합니다.

```
if nxt not in visited:
```

이미 방문한 정점이면 또 가 볼 필요가 없습니다.

```
visited.add(nxt)
```

큐에 넣기 전에 방문 처리합니다. 이 타이밍이 중요합니다.

```
queue.append(nxt)
```

다음 차례에 탐색할 수 있도록 큐 뒤쪽에 넣습니다.

6-4. BFS 손으로 따라가기

```
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}
```

단계	큐 상태	꺼낸 정점	새로 넣은 정점	방문 순서
시작	[1]	-	-	[]
1	[]	1	2, 3	[1]
2	[3]	2	4, 5	[1, 2]
3	[4, 5]	3	6	[1, 2, 3]
4	[5, 6]	4	-	[1, 2, 3, 4]
5	[6]	5	-	[1, 2, 3, 4, 5]
6	[]	6	-	[1, 2, 3, 4, 5, 6]

6-5. BFS를 실행하면

```
print(bfs(graph, 1))
```

```
[1, 2, 3, 4, 5, 6]
```

거리 1인 정점 2, 3이 먼저 나오고, 그 다음 거리 2인 정점 4, 5, 6이 나옵니다.

7. BFS와 최단 거리

가중치가 없는 그래프에서는 BFS가 최단 거리 문제와 아주 잘 맞습니다.

```
from collections import deque

def bfs_distance(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        now = queue.popleft()

        for nxt in graph[now]:
            if nxt not in distance:
                distance[nxt] = distance[now] + 1
                queue.append(nxt)

    return distance
```

7-1. 코드 해석

- `distance[start] = 0` : 시작 정점에서 시작 정점까지 거리는 0입니다.
- `distance[nxt] = distance[now] + 1` : 한 칸 이동했으니 거리가 1 늘어납니다.
- 한 번 거리값이 정해진 정점은 더 짧게 갈 방법이 BFS에서는 다시 나오지 않습니다.

7-2. 최단 거리 예시

```
graph = {
    1: [2, 3],
    2: [4],
    3: [4, 5],
    4: [6],
    5: [6],
    6: []
}

print(bfs_distance(graph, 1))
```

```
{1: 0, 2: 1, 3: 1, 4: 2, 5: 2, 6: 3}
```

8. DFS의 핵심 감각

DFS는 한 방향으로 갈 수 있을 때까지 깊게 들어갑니다. 막히면 돌아옵니다.

DFS를 한 줄로 말하면: **한 길을 끝까지 파고 들어가 본 뒤, 막히면 되돌아와 다른 길을 가는 방식**입니다.

8-1. DFS 기본 코드

```
def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for nxt in graph[start]:
        if nxt not in visited:
            dfs(graph, nxt, visited, order)

    return order
```

8-2. 코드 해석

if visited is None:

함수를 처음 호출했을 때만 빈 방문 집합을 만듭니다.

visited.add(start)

현재 정점을 방문 처리합니다.

order.append(start)

현재 정점을 방문 순서에 기록합니다.

for nxt in graph[start]:

현재 정점과 연결된 이웃을 하나씩 확인합니다.

dfs(graph, nxt, visited, order)

방문하지 않은 이웃이 있으면 그쪽으로 더 깊게 들어갑니다.

8-3. DFS 손으로 따라가기

```
graph = {
    1: [2, 3],
    2: [4],
    3: [5],
    4: [],
    5: []
}
```

1. 1 방문
2. 2로 이동
3. 4로 이동
4. 4에서 갈 곳이 없으니 돌아감
5. 2도 끝났으니 돌아감
6. 1의 다른 이웃 3으로 이동
7. 5로 이동

```
print(dfs(graph, 1))
```

```
[1, 2, 4, 3, 5]
```

9. DFS가 재귀로 돌아가는 원리

많이 헛갈리는 부분이라 코드보다 먼저 흐름을 잡는 게 중요합니다.

```
dfs(1)
-> dfs(2)
  -> dfs(4)
    <- 4 끝
  <- 2 끝
-> dfs(3)
  -> dfs(5)
    <- 5 끝
  <- 3 끝
<- 1 끝
```

재귀는 함수가 자기 자신을 다시 부르는 것이고, 그 호출들이 스택처럼 쌓였다가 끝나면 거꾸로 돌아옵니다.

10. DFS의 발견 시간과 종료 시간

CLRS에서는 DFS를 단순 방문에서 끝내지 않고, 각 정점의 시작 시점과 종료 시점을 기록합니다.

```
def dfs_with_time(graph):
    visited = set()
    discover = {}
    finish = {}
    time = 0

    def visit(node):
        nonlocal time
        time += 1
        discover[node] = time
        visited.add(node)

        for nxt in graph[node]:
            if nxt not in visited:
                visit(nxt)

        time += 1
        finish[node] = time

    for node in graph:
        if node not in visited:
            visit(node)

    return discover, finish
```

10-1. 왜 이런 시간이 필요한가

- 탐색 구조를 분석할 수 있습니다.
- 위상정렬과 연결됩니다.
- 간선 분류 같은 고급 개념의 기반이 됩니다.

11. BFS와 DFS를 비교해서 이해하기

항목	BFS	DFS
느낌	넓게 퍼짐	깊게 내려감
자료구조	큐	재귀 또는 스택
주요 강점	최단 거리	구조 분석
대표 응용	미로, 거리 계산	사이클, 위상정렬
시간복잡도	$O(V + E)$	$O(V + E)$

12. 코드로 둘을 같은 그래프에서 비교

```
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}

print("BFS:", bfs(graph, 1))
print("DFS:", dfs(graph, 1))
```

```
BFS: [1, 2, 3, 4, 5, 6]
DFS: [1, 2, 4, 5, 3, 6]
```

같은 그래프라도 방문 순서는 달라집니다. 이 차이를 보는 것이 중요합니다.

13. 위상정렬이 필요한 이유

작업에 순서가 있을 때 단순한 탐색만으로는 부족합니다. 먼저 해야 할 일을 앞에 놓아야 합니다.

- 선수 과목 순서
- 프로젝트 작업 순서
- 빌드 순서
- 의존성 설치 순서

14. DAG가 무엇인가

DAG는 Directed Acyclic Graph의 줄임말입니다.

- Directed: 방향이 있음
- Acyclic: 사이클이 없음

위상정렬은 DAG에서만 가능합니다.

기억법: 위상정렬은 "앞뒤 순서를 정하는 것"인데, 사이클이 있으면 누가 먼저인지 모순이 생겨서 순서를 만들 수 없습니다.

15. DFS 기반 위상정렬

```
def topological_sort(graph):
    visited = set()
    result = []

    def visit(node):
        visited.add(node)

        for nxt in graph[node]:
            if nxt not in visited:
                visit(nxt)

        result.append(node)

    for node in graph:
        if node not in visited:
            visit(node)

    result.reverse()
    return result
```

15-1. 코드 해석

`result.append(node)`

현재 정점의 모든 후속 작업을 먼저 처리한 뒤, 마지막에 현재 정점을 넣습니다.

`result.reverse()`

끝난 순서대로 쌓였기 때문에 뒤집어야 선행 작업이 앞에 오게 됩니다.

15-2. 예시 그래프

```
graph = {
    "파이썬 기초": ["자료구조"],
    "자료구조": ["알고리즘"],
    "알고리즘": ["그래프"],
    "그래프": ["위상정렬"],
    "위상정렬": []
}
```

```
print(topological_sort(graph))
```

```
['파이썬 기초', '자료구조', '알고리즘', '그래프', '위상정렬']
```

16. 위상정렬을 손으로 따라가기

1. 파이썬 기초 에서 시작
2. 자료구조 로 이동

3. 알고리즘으로 이동
4. 그래프로 이동
5. 위상정렬로 이동
6. 더 갈 곳이 없으니 위상정렬을 result에 넣음
7. 돌아오면서 그래프, 알고리즘, 자료구조, 파이썬 기초 순으로 넣음
8. 마지막으로 reverse 하면 선행 순서가 맞춰짐

17. 사이클이 있으면 왜 안 되는가

A -> B -> C -> A

이 구조에서는 A가 B보다 먼저여야 하고, B가 C보다 먼저여야 하고, C가 A보다 먼저여야 합니다. 모순입니다. 그래서 사이클이 있으면 위상정렬은 불가능합니다.

18. 사이클 감지까지 넣은 위상정렬

실전에서는 그래프에 사이클이 있을 가능성도 생각해야 합니다.

```
def topological_sort_with_cycle_check(graph):
    visited = set()
    visiting = set()
    result = []

    def visit(node):
        if node in visiting:
            raise ValueError("사이클이 존재해서 위상정렬이 불가능합니다.")
        if node in visited:
            return
        visiting.add(node)
        for nxt in graph[node]:
            visit(nxt)
        visiting.remove(node)
        visited.add(node)
        result.append(node)

    for node in graph:
        if node not in visited:
            visit(node)

    result.reverse()
    return result
```

18-1. 핵심 아이디어

- visiting : 현재 재귀 호출 경로 안에 있는 정점
- 다시 visiting 에 있는 정점을 만나면 순환이 있다는 뜻
- visited : 완전히 처리가 끝난 정점

19. Kahn 알고리즘도 알아두면 좋은가

위상정렬은 DFS 기반 말고도 진입 차수(indegree)를 쓰는 Kahn 알고리즘으로도 할 수 있습니다.

```
from collections import deque

def kahn_topological_sort(graph):
    indegree = {node: 0 for node in graph}

    for node in graph:
        for nxt in graph[node]:
            indegree[nxt] += 1

    queue = deque([node for node in graph if indegree[node] == 0])
    result = []

    while queue:
        now = queue.popleft()
        result.append(now)

        for nxt in graph[now]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)

    if len(result) != len(graph):
        raise ValueError("사이클이 존재합니다.")

    return result
```

19-1. 언제 유용한가

- 진입 차수 개념을 같이 익히고 싶을 때
- BFS 느낌으로 위상정렬을 보고 싶을 때
- 사이클 판별을 결과 길이로 쉽게 보고 싶을 때

20. BFS, DFS, 위상정렬 전체 코드 모음

```

from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        now = queue.popleft()
        order.append(now)

        for nxt in graph[now]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)

    return order

def bfs_distance(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        now = queue.popleft()

        for nxt in graph[now]:
            if nxt not in distance:
                distance[nxt] = distance[now] + 1
                queue.append(nxt)

    return distance

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for nxt in graph[start]:
        if nxt not in visited:
            dfs(graph, nxt, visited, order)

    return order

def topological_sort(graph):
    visited = set()
    result = []

    def visit(node):
        visited.add(node)

        for nxt in graph[node]:
            if nxt not in visited:
                visit(nxt)

        result.append(node)

    for node in graph:
        if node not in visited:
            visit(node)

```

```
visit(node)
```

```
result.reverse()
return result
```

21. 한 번에 실행해보는 예제

```
graph1 = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}

graph2 = {
    "파이썬 기초": ["자료구조"],
    "자료구조": ["알고리즘"],
    "알고리즘": ["그래프"],
    "그래프": ["위상정렬"],
    "위상정렬": []
}

print("BFS 방문 순서:", bfs(graph1, 1))
print("BFS 거리:", bfs_distance(graph1, 1))
print("DFS 방문 순서:", dfs(graph1, 1))
print("위상정렬:", topological_sort(graph2))
```

22. 출력 결과를 읽는 법

```
BFS 방문 순서: [1, 2, 3, 4, 5, 6]
BFS 거리: {1: 0, 2: 1, 3: 1, 4: 2, 5: 2, 6: 2}
DFS 방문 순서: [1, 2, 4, 5, 3, 6]
위상정렬: ['파이썬 기초', '자료구조', '알고리즘', '그래프', '위상정렬']
```

- BFS 방문 순서: 가까운 곳부터 확인했다는 뜻입니다.
- BFS 거리: 시작점에서 몇 단계 떨어져 있는지 보여줍니다.
- DFS 방문 순서: 한 길을 깊게 탄 흔적입니다.
- 위상정렬: 선행 작업이 먼저 오도록 정렬된 결과입니다.

23. 실수하기 쉬운 포인트 모음

23-1. 방문 처리를 너무 늦게 하는 실수

특히 BFS에서 큐에서 꺼낼 때 방문 처리하면 같은 정점이 여러 번 들어갈 수 있습니다.

23-2. 방향 그래프를 무방향처럼 저장하는 실수

위상정렬 문제는 대부분 방향 그래프입니다. 반대 방향을 자동으로 추가하면 안 됩니다.

23-3. 정점 번호 범위를 잘못 잡는 실수

문제가 1번부터 주는지, 0번부터 주는지 꼭 확인해야 합니다.

23-4. DFS 재귀 깊이 문제

정점 수가 매우 크면 파이썬에서 재귀 제한을 만날 수 있습니다. 그럴 때는 스택 구현을 고려합니다.

24. 연습 문제를 만났을 때 사고 순서

1. 그래프가 방향인지 무방향인지 확인한다.
2. 간선에 가중치가 있는지 확인한다.
3. 최단 거리인지, 단순 방문인지, 순서 문제인지 파악한다.
4. BFS인지 DFS인지 위상정렬인지 선택한다.
5. 그래프 저장 방식을 인접 리스트로 돌지 판단한다.

25. BFS를 떠올리는 문제 신호

- 최소 횟수
- 최단 이동
- 몇 단계 만에
- 동시에 퍼짐

26. DFS를 떠올리는 문제 신호

- 연결 요소 개수
- 사이클 존재 여부
- 한 경로를 끝까지 따라가 보기
- 위상정렬, SCC 같은 구조 분석

27. 위상정렬을 떠올리는 문제 신호

- 선행 관계
- 먼저 해야 하는 작업
- 순서를 정해야 함
- 모든 조건을 만족하는 배치

28. 초보자용 암기보다 이해 포인트

BFS

가까운 순서를 보장하려면 큐가 필요하다는 점을 이해하면 됩니다.

DFS

깊게 들어갔다가 되돌아오는 흐름을 이해하면 재귀가 덜 무섭습니다.

위상정렬

후속 작업을 먼저 끝내고 현재 작업을 넣은 뒤 뒤집는 흐름이 핵심입니다.

그래프 저장

처음에는 인접 리스트를 기본 형태로 익히는 것이 가장 실용적입니다.

29. 마지막 정리

1. 그래프는 연결 관계를 표현하는 자료구조입니다.
2. BFS는 큐를 이용해 가까운 정점부터 탐색합니다.
3. DFS는 재귀 또는 스택을 이용해 깊게 탐색합니다.
4. 위상정렬은 DAG에서 선행 관계를 지키는 순서를 만드는 방법입니다.
5. 코드를 외우기보다, 왜 그 자료구조를 쓰는지 이해하는 것이 더 중요합니다.

30. 다음 단계 추천

이 문서까지 이해했다면 다음에는 아래 방향으로 가면 좋습니다.

- BFS 문제 2~3개 직접 풀어보기
- DFS로 연결 요소 개수 세기
- 위상정렬 문제에서 사이클이 있는 경우 처리해보기
- 나중에는 다익스트라와 Union-Find로 확장하기

제일 좋은 복습은 직접 타이핑입니다. 문서를 읽고 끝내지 말고, 예제 그래프를 바꾸면서 출력 결과가 어떻게 달라지는지 꼭 확인해보면 좋습니다.